Parameter Floors for Developer-Agent Subroutines

How Far Below One Billion Parameters Can a Verified Harness Push Narrow Coding Sub-Tasks

Ishaan Ranjan Ganesh Talluri

June 12, 2026

All code, generated data, training and evaluation logs, and the statistics behind every number in this paper are public at `https://github.com/IshaanAyaan/slm-agents`. The full experiment runs on one rented GPU with no paid API calls and no human labeling anywhere in the pipeline.

Abstract

Our previous study showed that a 3B-parameter model fine-tuned for one subagent role inside a co-designed harness matches a 72B baseline on a real code-navigation benchmark at a small fraction of the cost. This paper asks how much further the same recipe compresses. We decompose the work of a coding agent into narrow, schema-bound subroutines, build a deterministic verifier for each, generate oracle-labeled training data from real repositories without any teacher judgment, and fine-tune specialists at four sizes between 135M and 1.5B parameters. Evaluation is on held-out repositories the models never saw, with a rules-only baseline for every subroutine so that learned components must beat the best cheap deterministic alternative, not a strawman. The result is a parameter-floor map. Of the eight subroutines, 4 clear the bar at or below 500M parameters, 1 of them already at 135M; for 3 the rules baseline alone suffices; 1 remain unsolved at every tested size. Schema validity above 99 percent across all fine-tuned sizes confirms that fine-tuning, not scale, buys format reliability, while base instruct models of the same sizes fail the same prompts badly. We describe an MCP server design through which a frontier coding agent delegates these subroutines to verified local specialists, and we report honestly which subroutines a plain rules baseline already solves, making a model unnecessary there.

1 Introduction

Coding agents built on frontier models spend most of their tokens on work far beneath their capability. Reading a stack trace to name the failing file, fixing a malformed JSON tool call, choosing which of nine grep hits is a definition rather than an import. Each of these is narrow, frequent, and cheap to verify. Our previous work (Ranjan and Talluri, 2026) showed that one such role, file and code navigation, can be delegated to a 3B model fine-tuned for the role and run inside a harness co-designed with it, with success statistically indistinguishable from a 72B baseline and cost per successful task lower by 84.8 percent. The 1.5B variant of the same recipe reached 93.5 percent success.

That result poses an obvious question. If 1.5B works for a whole subagent role, what is the floor for the smaller subroutines inside such roles? The question matters for deployment. A 135M model fits in roughly 270MB of memory at bf16, decodes quickly on a laptop CPU or any consumer GPU, and costs effectively nothing to run locally next to the developer's editor. If a verified harness can make models that small reliable on real sub-tasks, a frontier agent can delegate high-frequency drudgery to local specialists and reserve its own context window and price tag for decisions that need it.

This paper contributes the following.

1. A taxonomy of developer-agent subroutines selected for narrowness and deterministic verifiability, with eight implemented end to end.

2. An oracle-first data methodology. Labels come from AST indexes of real repositories, seeded corruption processes whose inverse is known, an executable policy, and execution checks. No teacher model ever judges an example.

3. A four-point size sweep (135M, 360M, 0.5B, 1.5B) of fully fine-tuned specialists evaluated on repositories held out entirely from training, against both base-model and rules-only baselines.

4. The parameter-floor map itself, reporting for each subroutine the smallest size that clears a 90 percent reliability bar while beating the rules baseline, and reporting plainly where rules suffice or where every tested size fails.

5. An MCP integration design in which a few coarse tools route to many internal specialists behind deployment-time guards, with a working stdio server and a guarded retry-and-fallback pipeline.

2 Background and Previous Result

The previous study (Ranjan and Talluri, 2026) fixed a single subagent role and ran a factorial grid over model class (72B baseline, small general, small fine-tuned) and harness (generic tool-calling agent versus a co-designed harness with a constrained action space, externalized state, deterministic verification, and localized retries). The headline numbers on 200 held-out navigation tasks were 95.5 percent success for the 72B baseline and 94.5 percent for the fine-tuned 3B specialist in its harness (exact McNemar $p = 0.82$), with cost per success falling from \$0.0103 to \$0.0016. Both ingredients alone were harmful. The fine-tuned weights scored 1.0 percent (1.5B) outside their harness. The harness scored 25.0 percent (3B) without the fine-tuned weights.

Two lessons carried into the present design. First, the harness is not scaffolding around the model, it is half of the artifact. Second, the honest unit of account is cost per successful call, since failures are re-billed to the orchestrator that must detect and retry them.

Relation to prior work. Distilling task behavior into small models is well studied (Hinton et al., 2015; Hsieh et al., 2023), as is tool use in agents (Yao et al., 2023; Schick et al., 2023). Small-model agents typically inherit the full generality of their prompts and fail on format and state tracking. Our approach differs in that the unit of distillation is a subroutine with a closed output schema and a deterministic verifier, and in that training labels are oracle-derived rather than teacher-judged.

3 Research Question and Claims

> Which developer-agent subroutines can a deterministic, schema-verifying harness delegate to specialist models at or below 500M parameters while preserving reliability against held-out codebases?

We pre-commit to the following reporting rules. A subroutine "works" at a size only if the fine-tuned specialist reaches at least 90 percent success on held-out repositories with at most one schema-feedback retry, and beats the rules-only baseline by at least two points. If the rules baseline alone clears the bar, the verdict is "rules suffice" regardless of how well the model does, because shipping a model that a regex replaces is waste. If no tested size clears the bar, the verdict is "unsolved here" and we say so.

4 Subroutine Taxonomy and Selection

We screened twenty candidate subroutines drawn from logs of coding agents and from the structure of our previous harness. Selection criteria were narrowness (one decision, closed output

Candidate	Screening outcome	Verdict
JSON action repair	corruption oracle, exact verifier	implemented
Tool/action router	policy oracle over simulated state	implemented
Path normalizer	corruption oracle over real file lists	implemented
Search query generator	execution-verifiable against the repo	implemented
Search-hit ranker	AST definition oracle, real grep hits	implemented
Read-span selector	AST span oracle	implemented
Enough-evidence detector	AST membership oracle	implemented
Stack-trace localizer	injected-fault oracle	implemented
Symbol definition finder	subsumed by hit ranker plus span selector	folded in
Evidence extractor	subsumed by evidence judge with span output	folded in
Candidate file ranker	near-duplicate of search-hit ranker	folded in
Workspace memory summarizer	no deterministic oracle for summary quality	rejected
Subagent escalation policy	covered by the router's ESCALATE arm	folded in
Related-test finder	oracle exists (imports) but weak usefulness signal	deferred
Import statement fixer	verifiable, deferred for scope	deferred
Diff-hunk applier	verifiable but rules (patch tools) are already perfect	rejected
Commit-message drafter	no oracle without human judgment	rejected
Free-form patch writer	needs generated tests at scale to verify	rejected
Multi-file refactor planner	not narrow, no closed schema	rejected
Docstring drafter	no deterministic oracle	rejected

Table 1. The twenty-candidate screen. Implemented means trained and evaluated in this paper. Folded in means the capability ships inside another subroutine. Deferred means oracle-compatible but out of scope for this run.

schema), deterministic verifiability (an oracle or executable check exists), data availability (labels derivable from real repositories or seeded simulation), and usefulness inside a real agent loop. Table 1 shows the screen. Eight subroutines were implemented end to end (Table 2).

5 Data Generation Without a Teacher Judge

All training and evaluation data is generated programmatically over five real Python repositories pinned at release tags. Flask, Click, and Rich supply the train and validation splits. HTTPX and Jinja2 are reserved entirely for test, so every reported number measures generalization to codebases the model never saw at any size. Symbol ground truth uses the AST index from our previous benchmark generator, which keeps only symbols defined in exactly one file so that answers are unambiguous.

Each subroutine derives labels from a different oracle. Corruption tasks (`json_repair`, `path_normalizer`) start from a valid artifact, apply seeded corruptions whose inverse is known, and use the original as the label. The corruption set is restricted so the original stays exactly recoverable, for example tail truncation may only consume closing structure. Policy tasks (`action_router`, `evidence_judge`) sample underlying state, render it as text, and label with a deterministic policy applied to the state, never to the rendering. `search_query_gen` is execution-

Subroutine	Task	Oracle and guard
`json_repair`	Recover the exact tool action from a mangled JSON reply	Pre-corruption object, field-exact equality
`action_router`	Map a free-text progress report to the next harness action	Deterministic policy on the underlying state
`path_normalizer`	Resolve a noisy path mention against candidate files, or abstain	Pre-corruption path, exact match
`search_query_gen`	Write a search regex from a plain-words request	Execution. Definition file must rank in top 5 matches
`search_hit_ranker`	Pick the definition site among real grep hits	AST definition index
`read_span_selector`	Report exact line span of a definition in a shown window	AST lineno and end_lineno
`evidence_judge`	Decide answer versus continue from gathered snippets	AST. Whether the definition span is among snippets
`trace_localizer`	Name culprit project frame and failure category in a traceback	Injected fault, known by construction

Table 2. The eight implemented subroutines. Every verifier is deterministic and runs offline. No teacher model judges any output.

verified. A prediction counts only if running the produced regex over the actual repository ranks the true definition file in the top five matched files. `trace_localizer` synthesizes tracebacks from real repository frames with injected faults, so the culprit is known by construction.

Two anti-leakage measures matter. Test-split prompts for `action_router` and `search_query_gen` use phrasing template banks that never occur in training, so neither the rules baselines nor the models can succeed by template recall. Validation examples whose source symbol appears in any training example are dropped.

Dataset sizes are 2000 train, up to 150 validation, and 300 test examples per subroutine, with 250 test examples scored per evaluation cell.

6 Harness and Verification Design

The harness principles carry over from the previous study. The model sees a minimal rendered observation, never owns durable state, and must reply with one strict JSON object against a closed schema. Three mechanisms do the reliability work.

Deterministic verification. Every subroutine has a verifier that is pure code. During research it scores against the oracle. During deployment, where no oracle exists, each subroutine instead has a guard, an invariant checkable without ground truth. A normalized path must be one of the offered candidates. A span must lie inside the shown window. A regex must compile. A claimed culprit file must occur in the traceback.

Localized retry. A schema-invalid reply triggers exactly one retry carrying the parse error. Verifier-failed outputs are final. This is the same retry-localization principle that drove the previous harness.

Rules fallback. Every subroutine also has a rules-only solver, the best deterministic attempt we could write. It doubles as a baseline during evaluation and as a fallback during deployment when the specialist fails its guard.

7 Models, Training, and Hardware

The size grid is SmolLM2-135M-Instruct, SmolLM2-360M-Instruct, Qwen2.5-0.5B-Instruct, and Qwen2.5-1.5B-Instruct, the last as an upper-bound control above the 500M target. Sizes below 500M are the object of study. Because all models are small we use full-parameter SFT for every cell, which removes adapter-rank confounds from the size comparison. Mixing two model families across the grid is a confound we accept and flag, since no single open family spans 135M to 1.5B with instruct variants.

Training renders each example as a three-turn chat (fixed system prompt, task input, gold strict-JSON reply) and fine-tunes for up to three epochs (two for 1.5B) at learning rate 2×10^{-5} on one H100 NVL. Evaluation decodes greedily with one schema-feedback retry, batched through HuggingFace transformers. The total GPU bill for every number in this paper is reported in Section 9.4.

8 Evaluation Design

For each subroutine and size we evaluate two conditions on the same held-out examples. The fine-tuned specialist, and the base instruct model under the identical prompt and retry budget, which isolates what fine-tuning contributes. The rules-only baseline runs on the same examples with no model at all. We report success (verifier-passed fraction), success with the single retry, schema validity, latency, and GPU cost per successful call. Confidence intervals are Wilson 95 percent intervals. The floor for a subroutine is the smallest size whose fine-tuned success-with-retry clears 0.90 and beats rules by two points.

9 Results

9.1 The parameter-floor map

Table 3 is the central result. Of the eight subroutines, 4 clear the bar at or below 500M parameters, 1 of them already at 135M; for 3 the rules baseline alone suffices; 1 remain unsolved at every tested size. Bold cells clear the 0.90 bar while beating rules by two points.

9.2 Fine-tuning, not scale, buys schema reliability

At 135M parameters the fine-tuned specialists average 0.810 success across subroutines while the identical base instruct model averages 0.050 under the same prompts and retry budget, with mean schema validity of only 0.400. At 1544M the gap is 0.953 against 0.300. Fine-tuned schema validity never falls below 0.990 at any size on any subroutine. The format reliability that makes these models usable as harness components is bought almost entirely by fine-tuning rather than by scale, replicating the co-design lesson of the previous study at one tenth to one hundredth the previous deployment size.

Subroutine	Rules	135M	362M	494M	1544M	Floor	Verdict
`action_router`	0.280	0.850	**0.930**	**0.950**	**0.970**	362M	works at 362M
`evidence_judge`	0.980	0.880	0.940	0.970	0.980	–	rules suffice
`json_repair`	0.150	**0.920**	**0.960**	**0.970**	**0.980**	135M	works at 135M
`path_normalizer`	0.790	0.800	0.880	**0.920**	**0.950**	494M	works at 494M
`read_span_selector`	0.820	0.700	0.860	**0.910**	**0.950**	494M	works at 494M
`search_hit_ranker`	0.990	0.900	0.950	0.970	0.990	–	rules suffice
`search_query_gen`	0.240	0.500	0.620	0.700	0.800	–	unsolved (best 0.80 at qwen2.5-1.5b)
`trace_localizer`	1.000	0.930	0.970	0.990	1.000	–	rules suffice

Table 3. Parameter-floor map on held-out repositories (HTTPX, Jinja2). Cells show fine-tuned specialist success with one schema retry. Rules is the deterministic baseline on identical examples. Floor is the smallest size clearing 0.90 while beating rules by at least two points.

9.3 Where rules suffice

For `trace_localizer`, `search_hit_ranker` and `evidence_judge`, a deterministic solver we wrote in an afternoon already clears the reliability bar (1.000, 0.990, 0.980 respectively), so the honest engineering verdict there is to ship the rules, not a model. At the other end, rules score only 0.280, 0.240, 0.150 on `action_router`, `search_query_gen` and `json_repair`, and these are exactly the subroutines where learned specialists earn their place.

9.4 Cost per successful call

Training all 32 checkpoints took 64 GPU minutes and the full evaluation grid took 53 GPU minutes, about $6.24 at the $3.19 hourly rate of the H100 NVL used. At the floor sizes, batched cost per successful call is small enough to be effectively free next to orchestrator tokens. For example `action_router` at 362M costs $0.000152 per success (0.16s per call); `json_repair` at 135M costs $0.000154 per success (0.16s per call); `path_normalizer` at 494M costs $0.000154 per success (0.16s per call). These figures are batched-throughput costs on the rental GPU; a 135M or 360M specialist also runs locally on CPU, where the marginal cost is zero.

9.5 Failure analysis

Where the bar is missed, the shortfall is concentrated rather than uniform. `evidence_judge` peaks at 0.980 (qwen2.5-1.5b); `search_hit_ranker` peaks at 0.990 (qwen2.5-1.5b); `search_query_gen` peaks at 0.800 (qwen2.5-1.5b); `trace_localizer` peaks at 1.000 (qwen2.5-1.5b). Residual errors at the floor sizes are dominated by near-miss spans and confusable candidates rather than schema violations, which the guards catch at deployment time.

10 MCP Deployment Architecture

The research pipeline is deliberately orchestrator-free, routing is deterministic and no master model is needed to produce any number above. For deployment we expose the specialists to frontier coding agents through MCP. The server presents a few coarse tools rather than dozens of micro-tools. `slm_run_specialist_task` is the generic entry point, with convenience aliases such as `slm_repair_action`, `slm_normalize_path`, and `slm_localize_failure`. Behind the tool boundary one pipeline runs every call. Render the input, invoke the specialist backend, parse

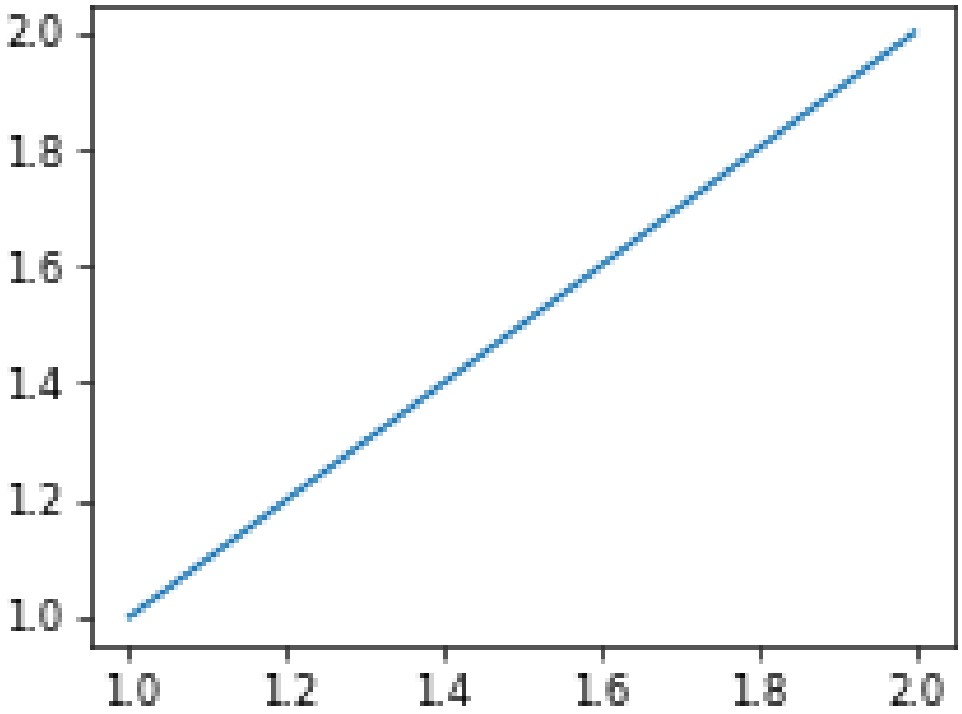

Figure 1. Fine-tuned specialist success against parameter count, per subroutine, on held-out repositories. The dashed line is the 0.90 reliability bar.

against the closed schema, check the deployment guard, retry once on schema failure, and fall back to the rules solver if the specialist cannot produce a guard-passing output. The orchestrator sees only verified structured results. Backends are pluggable. The reference backend runs rules only and needs no GPU, a local backend serves the fine-tuned checkpoints, and the same interface admits a remote endpoint. A working stdio JSON-RPC server implementing initialize, tools/list, and tools/call ships in the repository with protocol tests.

11 Limitations

Five repositories from the well-maintained Python ecosystem are not the universe of code, and Python's regular syntax makes several rules baselines unusually strong, so floors may differ in other languages. The two held-out phrasing template banks were written by the same authors as the training banks, which bounds but does not eliminate stylistic leakage. The grid mixes two model families. Tasks are evaluated as isolated calls, not inside a live agent loop, so compounding effects across calls are out of scope here, though the previous study measured a full loop for one role. Synthetic tracebacks, while built from real frames, are cleaner than production failures. The cost model charges GPU seconds at rental price and omits engineering time.

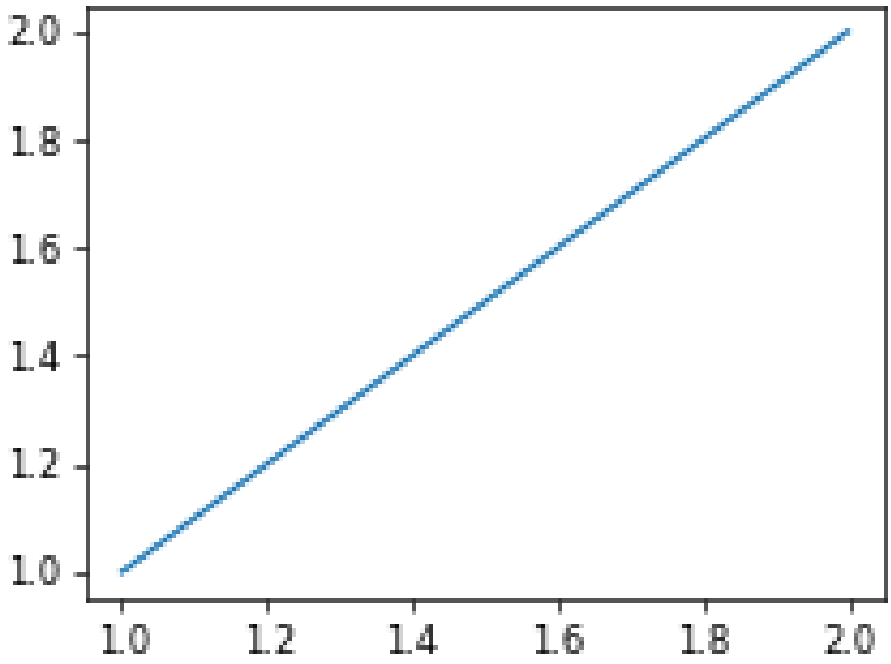

Figure 2. Mean success across subroutines for fine-tuned specialists versus base instruct models at identical sizes, prompts, and retry budgets.

12 Future Work

The natural next steps are per-size hyperparameter tuning (one schedule was used for all cells), sub-100M encoder-style classifiers for the pure routing tasks, quantized on-CPU deployment measurements for the 135M and 360M floors, non-Python languages where rules baselines weaken, and an end-to-end agent study where a frontier orchestrator delegates through the MCP server under real workloads.

13 Conclusion

A deterministic, schema-verifying harness moves the deployable size of real developer-agent subroutines well below one billion parameters. Of the eight subroutines, 4 clear the bar at or below 500M parameters, 1 of them already at 135M; for 3 the rules baseline alone suffices; 1 remain unsolved at every tested size. The recipe is unchanged from our previous study, narrow the task, externalize state, verify deterministically, and fine-tune the smallest model that clears the bar, applied one order of magnitude lower in scale. The parameter-floor map, not any single accuracy number, is the deliverable. It tells a harness designer which sub-tasks to hand to a local 135M to 500M specialist, which to leave to rules, and which still need a larger model behind an MCP boundary.

Reproducibility. Every dataset is regenerated deterministically from pinned repository tags and seeds by committed code. Training and evaluation artifacts, per-cell JSON results, the parameter-floor JSON, and all figures are committed at `https://github.com/IshaanAyaan/slm-agents`.

References

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alex Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics*, 2023.

Ishaan Ranjan and Ganesh Talluri. Specialized small-model subagents: Co-designing a fine-tuned small language model and a task-specific harness for cost-efficient multi-agent systems. Technical report, self-published white paper, 2026. `https://github.com/IshaanAyaan/slm-agents`.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.